

PA1: Implementing xv6 User Commands and System Calls
50 points

For this assignment, you will implement the UNIX utility `uptime` for xv6. You will also implement a `time` command for xv6 that takes a command and the command's arguments, executes the command, and, when the command has finished, outputs the CPU time, elapsed (also called wallclock) time, and the percentage of the CPU the process had while it was in the system. Your `time` command output will be similar to the `/bin/time` command in Linux, except that you do not need to keep track of the user and system portions of the CPU time separately, nor do you need to report memory usage information. Type '`man time`' on a Linux system for more information about the Linux `time` command.

Implementing the `time` command will help you to collect data to use for evaluating the scheduler you will implement for PA2. It's also useful for users to be able to find out the CPU time and elapsed time for their program. In this assignment, you will also learn how to implement a new system call, `wait2()`, which you will need to get CPU time.

Important note: For this programming assignment and the next one, we want to use just one CPU so that all processes are scheduled on the same CPU. To make this happen, in the Makefile, change `CPUS := 3` to `CPUS := 1`.

GitHub instructions:

1. Please create a GitHub account if you don't already have one. Then fork the xv6 GitHub repository at [https://github.com/mit-pdos/xv6-riscv/](https://github.com/mit-pdos/xv6-riscv) and make a `pa1-syscalls` branch for this assignment.
2. Point your existing local clone at your fork instead of upstream:

```
git remote set-url origin https://github.com/<your-GitHub-username>/xv6.git
```
3. Branch and push to your fork:

```
git checkout -b pa1-syscalls  
git push -u origin pa1-syscalls
```

Task 1. (5 pts) Implement the UNIX utility `uptime` for xv6. Your `uptime` command should print the number of clock ticks since system start. A tick is a notion of time defined by the xv6 kernel, namely the time between two interrupts from the timer chip. Your solution should be in the file `user/uptime.c`.

Some hints:

- Use the system call `uptime()`.
- See `kernel/sysproc.c` for the xv6 kernel code that implements the `uptime` system call (look for `sys_uptime`), `user/user.h` for the C definition

of `uptime` callable from a user program, and `user/usys.S` for the assembler code that jumps from user code into the kernel for `uptime`.

- Make sure `main` calls `exit()` in order to exit your program.
- Add your `uptime` program to `UPROGS` in `Makefile`; once you've done that, `make qemu` will compile your program and you'll be able to run it from the `xv6` shell.

Run the program from the `xv6` shell:

```
$ make qemu
...
init: starting sh
$ uptime
up xxxx clock ticks
$
```

Task 2. (10 pts) Implement a `time1` command that reports just elapsed time.

Your `time1` program should output a usage message and exit if the user does not enter a command to be timed. For this task, you will need to do the following:

- Handle the command-line arguments input by the user.
- Assemble the arguments to pass to `exec()`.
- Use `fork()` to create a child process.
- Use `exec()` in the child to execute the program passed as a command to `time1`.
- Call `wait()` in the parent to wait until the child finishes.
- Get the elapsed time by calling the `uptime()` system call in the parent before forking the child and after the `wait` returns.

Sample output for Task2 follows below.

```
$ time1 matmul
Time: 28 ticks
elapsed time: 28 ticks
$ time1 matmul & time1 matmul &
$ Time: 29 ticks
elapsed time: 30 ticks
Time: 54 ticks
elapsed time: 56 ticks
$ time1 sleep 10
elapsed time: 10 ticks
```

Task 3. (10 points) Keep track of how much cputime a process has used.

Add a cputime field to struct proc in kernel/proc.h. Initialize this field to zero when the process is created in allocproc() in kernel/proc.c. Increment this field every time the process uses up its time slice (see usertrap() and kerneltrap() in kernel/trap.c).

Task 4. (10 points) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

- Create a file kernel/pstat.h with the following contents:
 struct rusage {
 uint cputime;
 };
Add the following line to user/user.h so that you can use struct rusage in user code:
 struct rusage;
- Create a system call with the following prototype (put this in user/user.h):
 int wait2(int*, struct rusage*);
- Add an entry for wait2() in user/usys.pl
- Add wait2 system call information to kernel/syscall.h and kernel/syscall.c
- Implement sys_wait2() in kernel/sysproc.c and kwait2() in kernel/proc.c so that sys_wait2() returns child process status and rusage, with rusage currently being only cputime. You can copy sys_wait() to sys_wait2() and kwait() to kwait2() and add the necessary code to return the second argument. Add the kernel kwait2() prototype to defs.h so that is visible outside proc.c. Add #include "pstat.h" to proc.c so that your kernel kwait2() can use struct rusage.

Task 5. (10 points) Implement the time command so that it calls wait2() to get cputime and outputs elapsed time, CPU time, and %CPU. Test your program with some different inputs and show the results. You are given matmul.c and sleep.c programs that you can add to the xv6 user programs and use for tests.

Sample output for Task 5 follows below.

```
$ matmul
Time: 35 ticks
$ time matmul
Time: 36 ticks
elapsed time: 36 ticks, cpu time: 36 ticks, 100% CPU
$ time matmul &
$ Time: 69 ticks
elapsed time: 70 ticks, cpu time: 35 ticks, 50% CPU
Time: 68 ticks
elapsed time: 69 ticks, cpu time: 35 ticks, 50% CPU
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
```

Task 6. (5 points) Summarize what you learned from doing this assignment. Describe any difficulties you encountered and if/how you overcame them.

Extra Credit Task. (5 points) Discuss limitations of our time command.

Turn-in procedure and Grading

Please use the accompanying template for your report. Convert your report to PDF format and push your pa1-syscalls branch with your code and report to your xv6 GitHub repository by the due date. Also turn in the assignment on Teams with the URL of your GitHub repo by the due date. Give access to your repo to the instructor and TA.

This assignment is worth 50 points. The breakdown of points is given in the task descriptions above. The points for each task will be evaluated during a one-on-one demo with the instructor or the TA, based on the correctness of your code, proper coding style and adequate comments, and the section of your report for that task.

You may discuss the assignment with other students, but do not share your code. Your code and lab report must be your own original work. Any resources you use should be credited in your report. If we suspect that code has been copied from an online website or GitHub repo, from a book, or from another student, or generated using AI without giving credit and describing in detail how AI was used, we will turn the matter over to the Office of Community Standards for adjudication.